Course Name:	Competitive Programming Lab	Semester:	IV
Date of Performance:	30/ 03/ 2026	DIV/ Batch No:	B2
Student Name:	Ashwera Hasan	Roll No:	16010124107

Experiment No: 6

Title: Graph Algorithms

Aim of the Experiment: To understand and use graph algorithms to solve the given problem

Objective of the Experiment:
To model and solve application-driven computational problems using graphs.

COs to be achieved:

CO 2 Model and solve computational problems using graphs, by selecting appropriate graph representations and applying graph-based reasoning to real applications.

Theory:

In computational problem-solving, a graph is a mathematical structure used to model pairwise relations between objects. A graph G = (V, E) consists of a set of vertices (or nodes) V and a set of edges E that connect these vertices. Graphs are essential for modeling real-world networks, such as city maps, social media connections, and computer networks.

Some applications of graphs in various domains:

Sr. no	Domain / Industry	Key Algorithms Used	Real-World Application
1	**Navigation & Mapping (Google Maps, Uber)**	Dijkstra's Algorithm, A* Search, Bellman-Ford	Calculating the fastest or shortest delivery route, estimating ETA.
2	**Social Networks (LinkedIn, Instagram)**	Breadth-First Search (BFS), Strongly Connected Components (Tarjan's/Kosaraju's)	"People you may know" recommendations, finding degrees of separation, identifying communities.
3	**OS & Task Scheduling (Linux, npm, University)**	Topological Sorting, Depth-First Search (DFS), Kahn's Algorithm	Resolving software package dependencies, CPU process scheduling, course prerequisites.
4	**Computer Networks**	Minimum Spanning Tree (Kruskal's / Prim's), Dijkstra's	Routing data packets efficiently, designing LAN cable layouts with

	(Internet, Telecom)	(OSPF protocol)	minimal wire length.
5	**E-Commerce & Streaming** **(Amazon, Netflix)**	Bipartite Matching, Graph Traversal (DFS / BFS)	Recommending products or movies based on user behavior ("Users who bought X also bought Y").

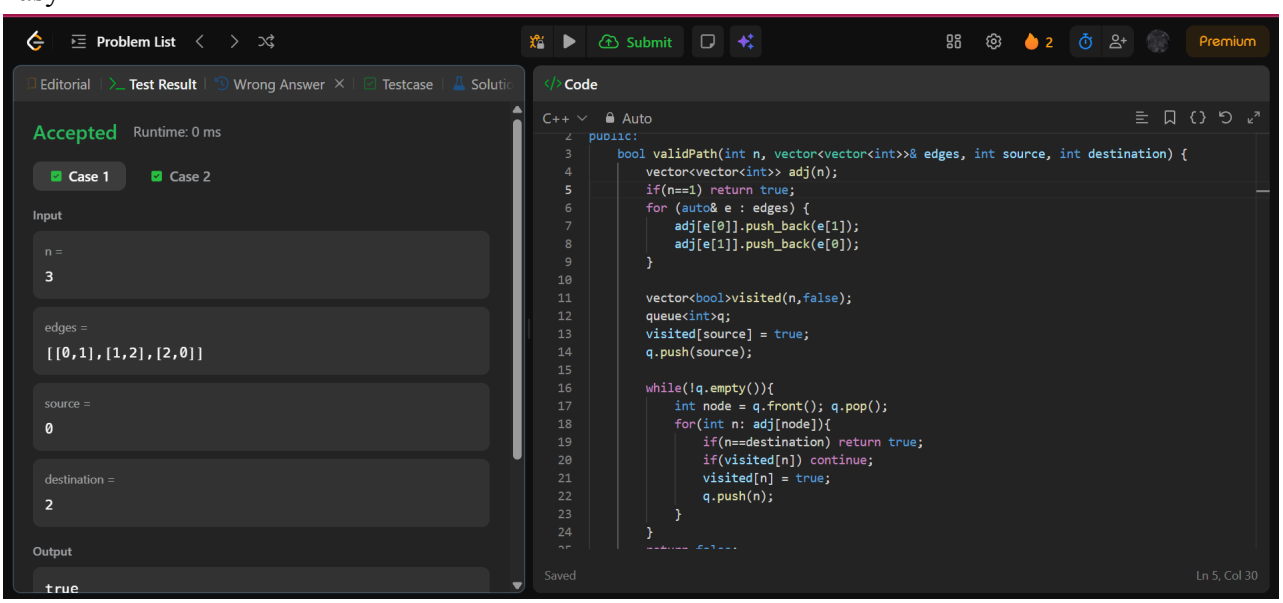
-

Problem Statements:

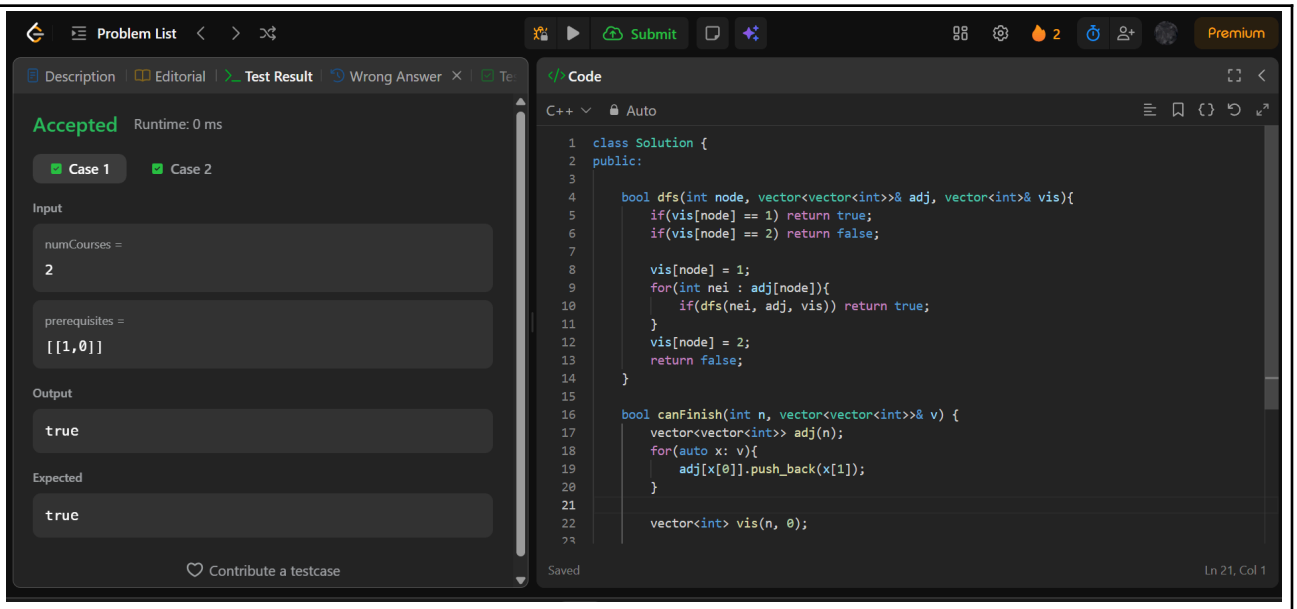
Difficulty (as per lab)	Problem	LeetCode mapping
Easy	Path checking problem	LC 1971 – Check if path exists
Medium	Topological sorting	LC 210 – Course Schedule II
Difficult	Cycle detection (directed)	LC 207 – Course Schedule

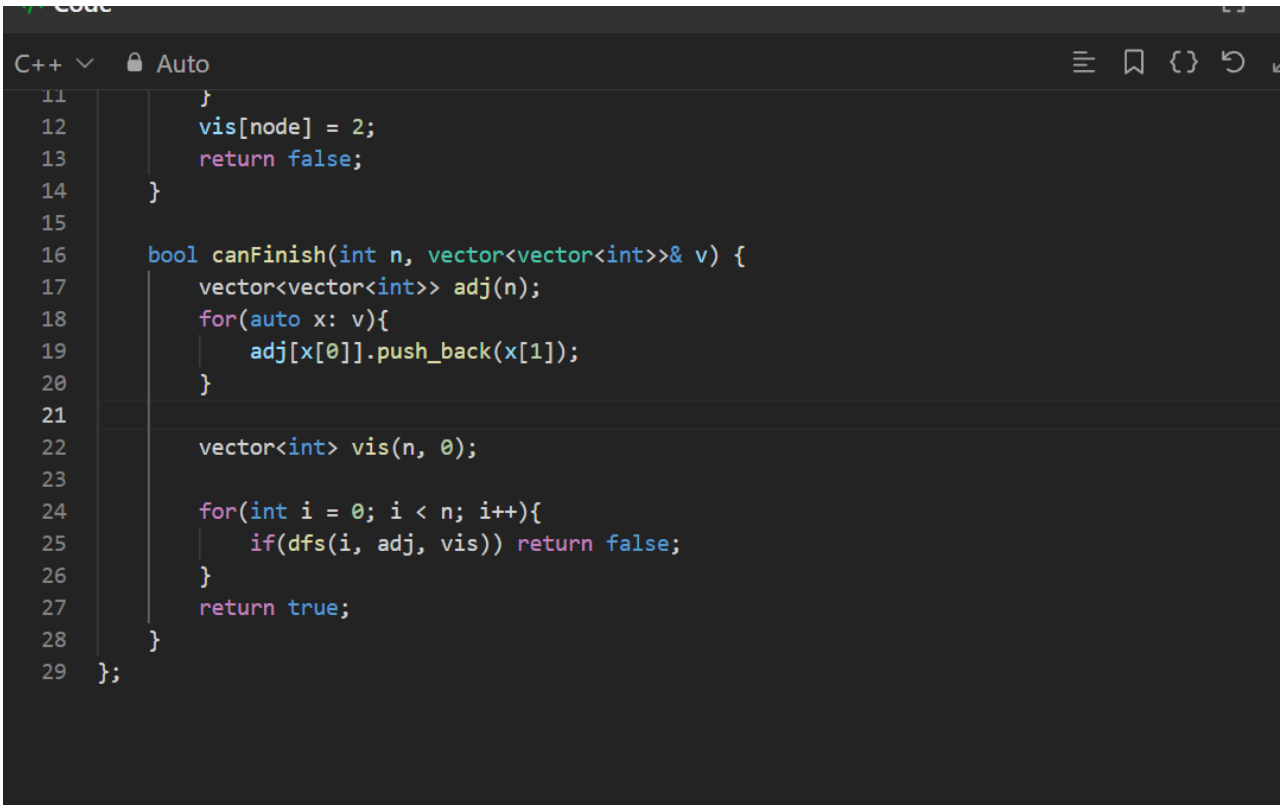
Code :

Easy

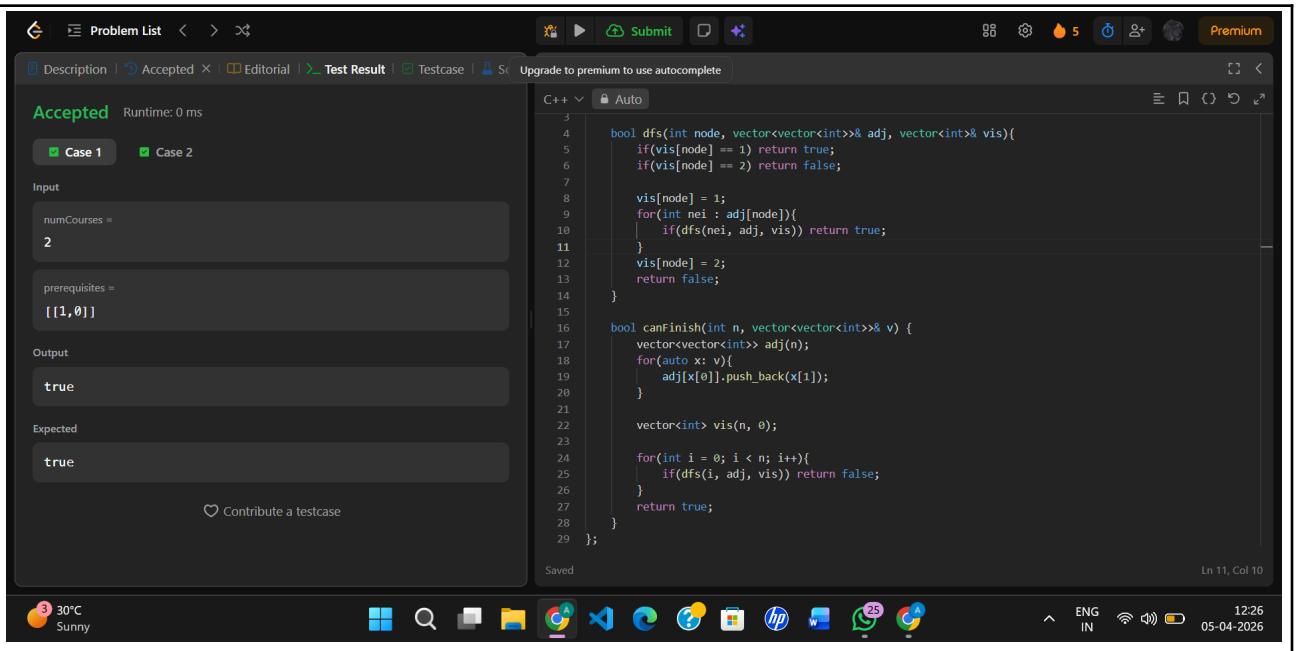


Medium:


Problem List
Submit
Premium
Description
Editorial
Test Result
Wrong Answer
Code
C++
Auto
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true
Contribute a testcase
class Solution {
public:
bool dfs(int node, vector<vector<int>>& adj, vector<int>& vis){
if(vis[node] == 1) return true;
if(vis[node] == 2) return false;
vis[node] = 1;
for(int nei : adj[node]){
if(dfs(nei, adj, vis)) return true;
}
vis[node] = 2;
return false;
}
bool canFinish(int n, vector<vector<int>>& v) {
vector<vector<int>> adj(n);
for(auto x: v){
adj[x[0]].push_back(x[1]);
}
vector<int> vis(n, 0);
Saved
Ln 21, Col 1

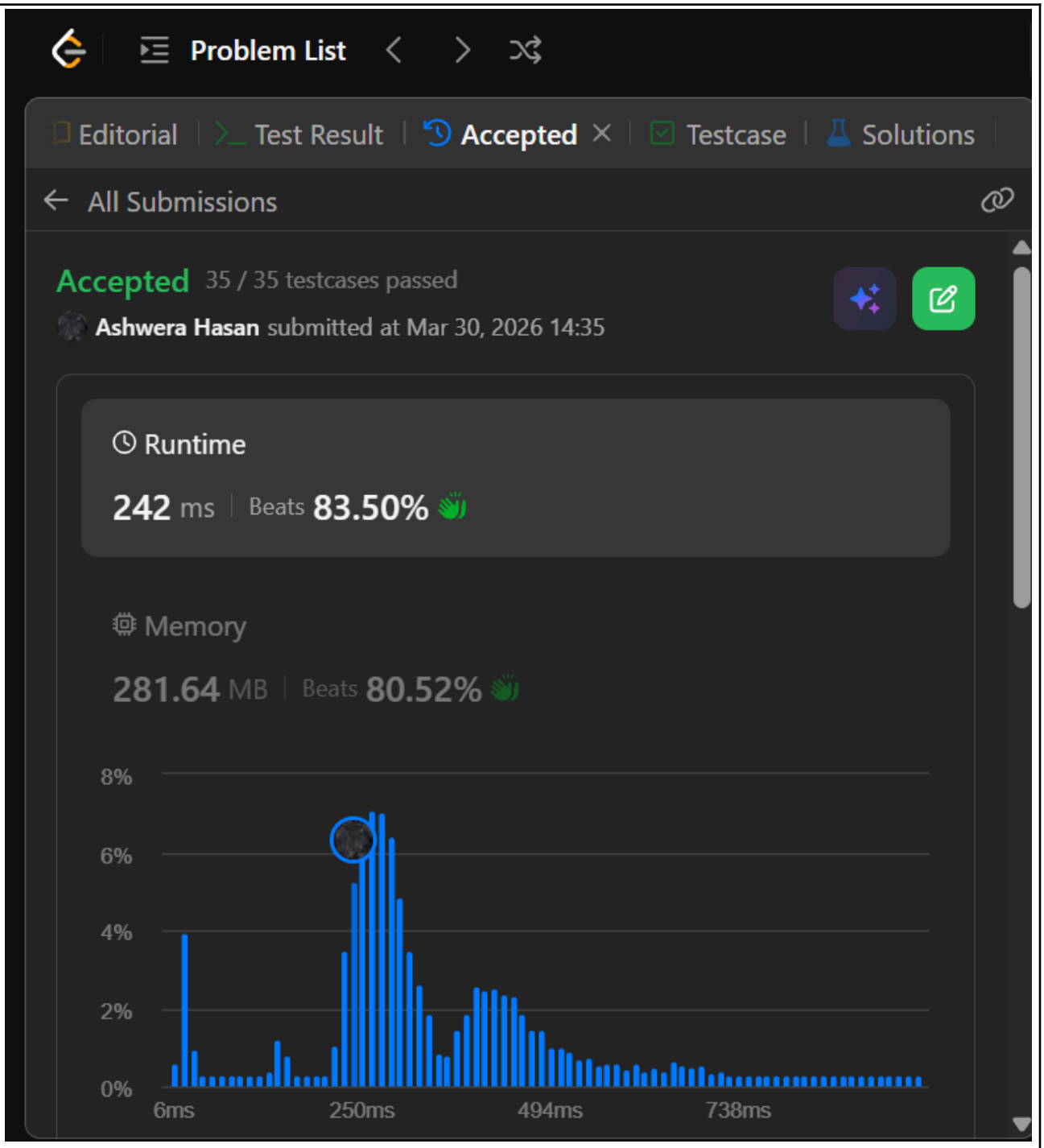

C++
Auto
}
vis[node] = 2;
return false;
}
bool canFinish(int n, vector<vector<int>>& v) {
vector<vector<int>> adj(n);
for(auto x: v){
adj[x[0]].push_back(x[1]);
}
vector<int> vis(n, 0);
for(int i = 0; i < n; i++){
if(dfs(i, adj, vis)) return false;
}
return true;
}
};

Hard:

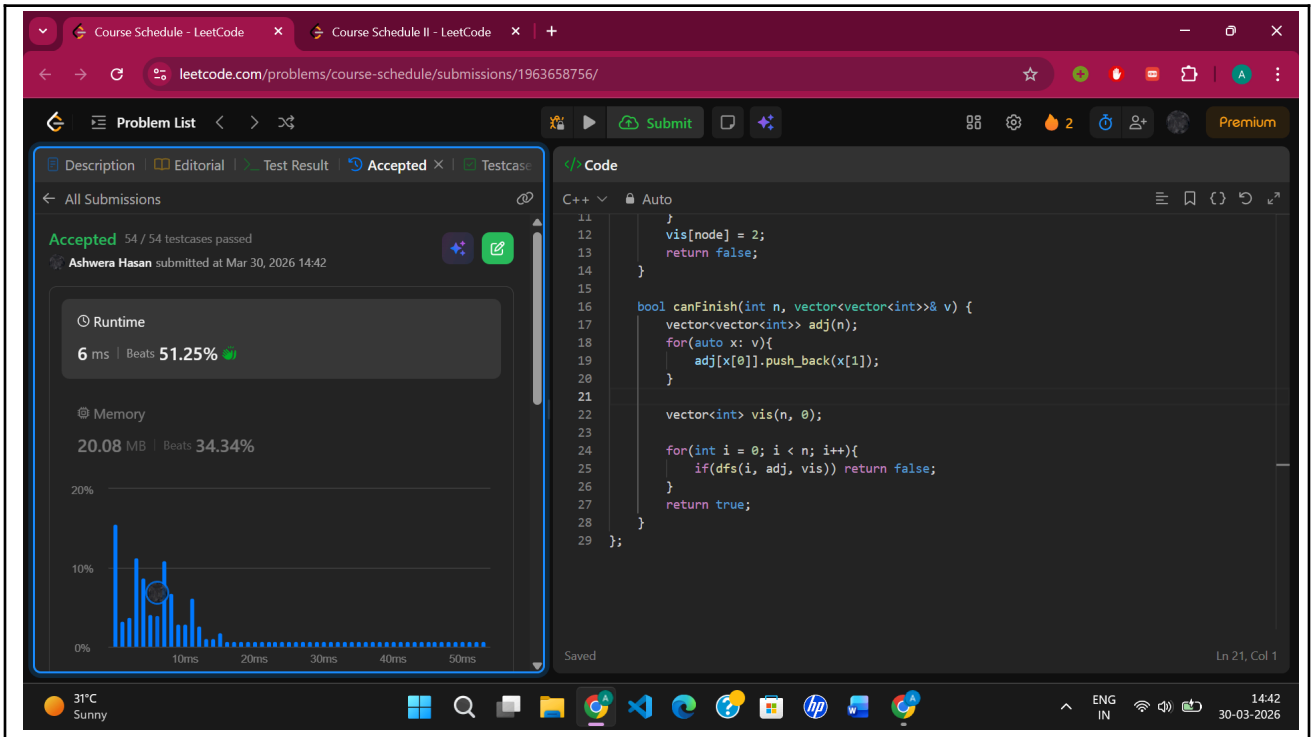

Problem List
Submit
Premium
Description
Accepted
Editorial
Test Result
Testcase
Upgrade to premium to use autocomplete
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true
Contribute a testcase
C++
Auto
bool dfs(int node, vector<vector<int>>& adj, vector<int>& vis){
if(vis[node] == 1) return true;
if(vis[node] == 2) return false;
vis[node] = 1;
for(int nei : adj[node]){
if(dfs(nei, adj, vis)) return true;
}
vis[node] = 2;
return false;
}
bool canFinish(int n, vector<vector<int>>& v) {
vector<vector<int>> adj(n);
for(auto x: v){
adj[x[0]].push_back(x[1]);
}
vector<int> vis(n, 0);
for(int i = 0; i < n; i++){
if(dfs(i, adj, vis)) return false;
}
return true;
}
};
Saved
Ln 11, Col 10
30°C
Sunny
ENG
IN
12:26
05-04-2026

Output:

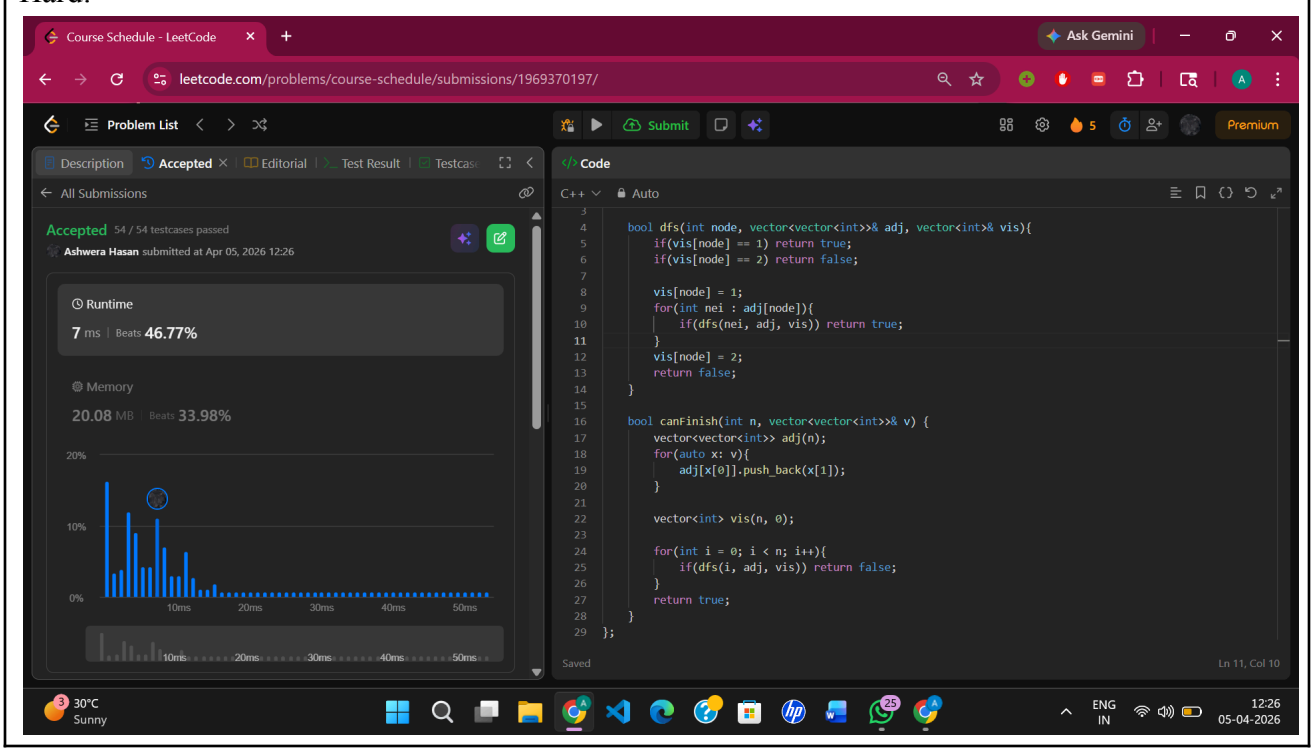
Easy:


Problem List
Editorial
Test Result
Accepted
Testcase
Solutions
All Submissions
Accepted 35 / 35 testcases passed
Ashwera Hasan submitted at Mar 30, 2026 14:35
Runtime
242 ms | Beats 83.50%
Memory
281.64 MB | Beats 80.52%
8%
6%
4%
2%
0%
6ms
250ms
494ms
738ms

Medium:



Hard:



Post Lab Subjective/Objective type Questions:

1. Both Depth-First Search (DFS) and Breadth-First Search (BFS) can successfully count the connected components. However, if a problem asks you to find the *shortest path* from an entry point to an exit point in an unweighted 2D maze, which traversal algorithm must you use and why?
   For finding the shortest path from an entry to an exit point in an unweighted 2D maze, we must use Breadth-First Search (BFS). BFS guarantees the shortest path in an unweighted graph (or grid, in the case of a maze) because it explores the graph level by level.
2. What is the maximum number of vertices (approximate) where an Adjacency Matrix is safe to use in standard CP platforms?
   Ideally a vector should stay within 1e8 limits. An adjacency matrix takes v^2 space. So the maximum number of vertices should be of the order 1e4, and it can safely go up to 5000-7000. 5000^2 is 25000000.

Conclusion:

Graph theory is a very important technique to understand and work around with CP problems where elements are interdependent on each other. By representing that data in the form of a connected graph, we can employ traversal techniques and other graph theory mechanisms to produce quick outputs.

Signature of faculty in-charge with Date: